

面向编码代理的 UNG 组内 PG 构建 GPU 并行化实施方案

1. 概述与目标

本文档为编码代理（Coding Agent）提供在 UNG (Unified Navigating Graph) 算法的构图阶段，针对同标签组内的 Proximity Graphs (PG) 生成步骤的 GPU 并行优化方案。文档旨在提供一套清晰、严谨且可直接转换为代码实现的工程指南。

1.1. 任务背景与在 UNG 中的位置

在 UNG 框架中，图的构建分为三个核心步骤：

- 构建标签导航图 (LNG)**：编码标签集之间的层级关系。
- 构建组内邻近图 (PG)**：在每个具有相同标签集的向量分组内部，构建一个近似最近邻（ANN）图。这是本方案优化的核心。
- 生成跨组边**：依据 LNG 的指引，在不同组的 PG 之间建立连接。

本方案聚焦于上述第 2 步，即如何利用 GPU 的大规模并行计算能力，高效地为**每一个标签组**生成高质量的 PG。

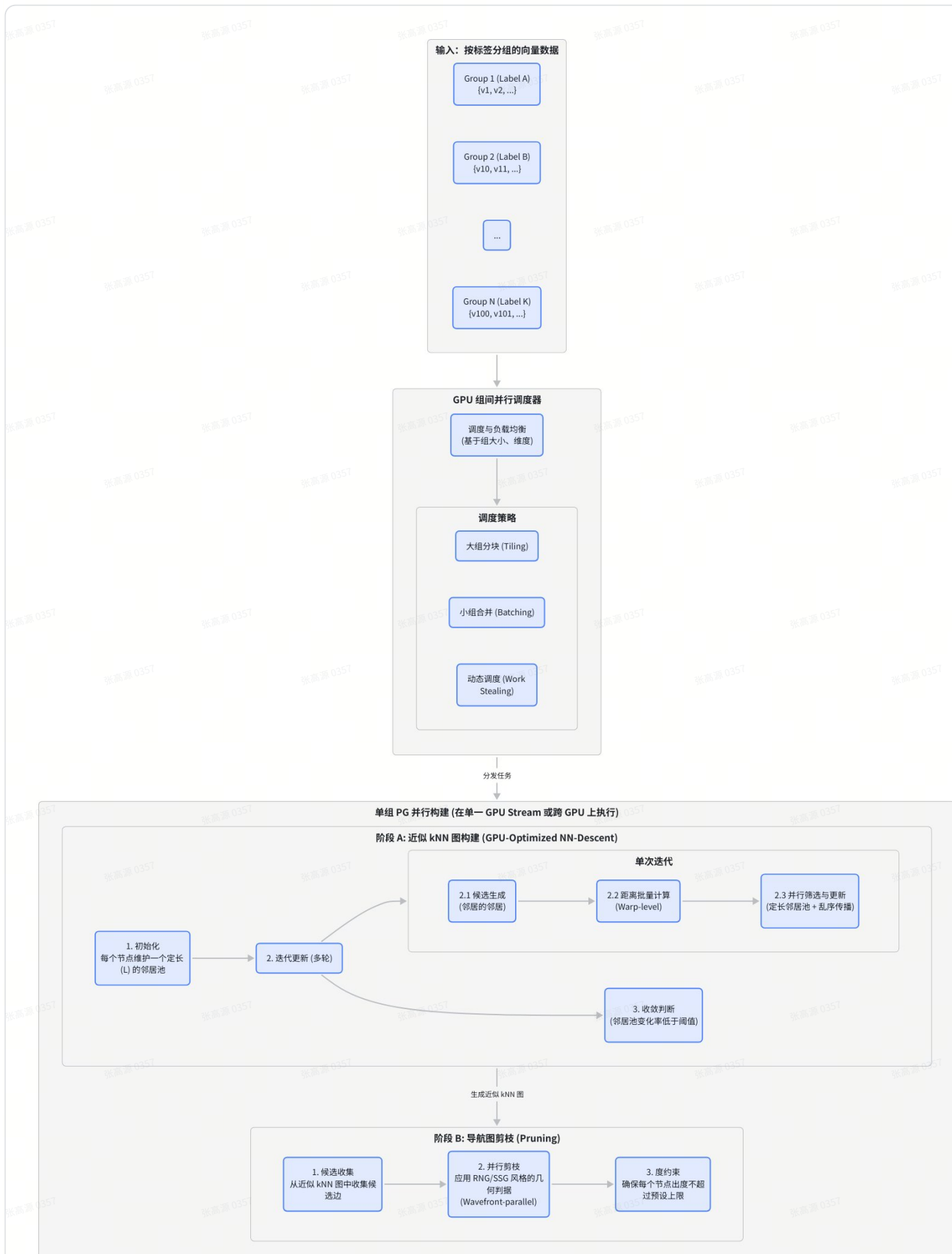
1.2. 核心目标与度量指标

优化方案的核心目标是在保证 PG 质量的前提下，最大化构图速度并提升 GPU 资源利用率。

度量指标	描述	优化目标
recall@k	在生成的 PG 上进行搜索的 top-k 查询召回率。这是衡量图质量的关键指标。	维持高水平（与 CPU 基线相当）
构图耗时 (Build Time)	完成所有组 PG 构建所需的总时间。	最小化
GPU 利用率	在构图期间，GPU 计算单元的繁忙时间占比。	最大化
显存占用 (Memory Footprint)	构图过程中峰值显存使用量。	可控且高效

2. 组内 PG 的两阶段 GPU 并行方案

为适配 GPU 的执行模型，我们将每个组内 PG 的构建过程分解为两个主要阶段。该方案借鉴了 GRNDA 和 Scalable Graph Indexing using GPUs 等文献中成熟的 GPU 图构建思想。





2.1. 阶段 A：近似 kNN 图构建

此阶段的目标是快速生成一个近似 kNN 图，它是一个比最终 PG 更稠密的中间结构。我们采用一种 GPU 优化的 NN-Descent 变体算法。

- **核心思想**：通过迭代式的“邻居的邻居”传播，并行地为每个节点优化其邻居列表。
- **关键特征**：
 - **定长邻居池**：每个节点维护一个固定长度为 L ($L > k$) 的邻居池，避免了动态内存分配的开销和不规则访存。
 - **乱序邻居传播 (Disordered Neighbor Propagation)**：在更新邻居时，随机顺序评估候选邻居对，打破了并行更新中的对称性，有效避免了“过早收敛”到局部最优的问题。
 - **迭代收敛**：当一轮迭代中邻居池的变化量（更新的邻居数量）低于预设阈值时，认为图结构已初步收敛，阶段 A 结束。

2.2. 阶段 B：可导航图剪枝

此阶段接收阶段 A 生成的近似 kNN 图，并将其剪枝 (Prune) 成一个更稀疏、出度受控且具有良好导航性的最终 PG。

- **核心思想**：并行地移除冗余边，同时保留对图连通性和“小世界”特性至关重要的长短边。
- **关键特征**：
 - **RNG/SSG 风格剪枝**：采用相对近邻图 (RNG) 或其变种 (如 SSG) 的几何判据。对于一条边 (u, v) ，如果存在一个公共邻居 w ，使得 $\text{dist}(u, w) < \text{dist}(u, v)$ 且 $\text{dist}(v, w) < \text{dist}(u, v)$ ，则 (u, v) 很可能是一条冗余的短边，可以被剪枝。
 - **并行筛选**：利用 Wavefront-parallel 等计算模型，在 GPU 上高效并行处理具有数据依赖的剪枝决策。
 - **度约束 (Degree Bounding)**：在剪枝后，强制保证每个节点的出度不超过一个预设的最大值 $D_{\max}$ ，以生成最终的稀疏图。

3. 数据结构与内存布局

高效的 GPU 计算依赖于精心设计的、对访存友好的数据结构。所有数据结构都应在 GPU 显存中分配。

3.1. 核心数据结构

结构名称	维度/布局	描述	GPU 优化考虑
顶点向量数据 (Vectors)	<code>N x D</code> (SoA 优先)	N 个 D 维向量。推荐使用 Structure of Arrays (SoA) 布局，即 <code>(dim_1, dim_2, ..., dim_D)</code> ，每个维度是一个连续数组。	访存合并 ：SoA 布局使得在计算距离时，一个 warp 内的线程可以访问连续的内存地址，触发合并访存，极大提升带宽利用率。
定长邻居池 (Neighbor Pool)	<code>N x L</code> (二维数组)	每个顶点关联一个长度为 L 的数组，存储邻居的索引 (ID)。	规整访存 ：固定长度消除了动态分配和指针追逐。warp 内线程访问不同顶点的邻居池时，地址具有固定偏移，易于优化。
邻居距离缓存 (Distance Cache)	<code>N x L</code> (二维数组)	与邻居池对应，存储每个邻居与当前顶点的距离。	数据局部性 ：将距离和邻居索引并置存放或采用 AoSoA 结构，可以提高缓存命中率。
候选缓冲 (Candidate Buffer)	<code>N x C</code> (线程/Warp私有)	在迭代更新时，临时存放新生成的候选邻居及其距离。C 为候选规模。	共享内存/寄存器 ：将此缓冲区放置在共享内存或寄存器中，可实现超高速的读写，避免访问全局显存。
图邻接表示 (Adjacency Rep.)	<code>N x D_max</code> (CSR/块结构)	最终剪枝后的 PG 的邻接列表， <code>D_max</code> 为最大出度。	压缩存储 ：使用压缩稀疏行 (CSR) 格式或固定大小的块结构存储最终的稀疏图，以节省显存。

3.2. 邻居池更新原子性策略

在并行更新邻居池时，为避免写冲突，我们不使用锁 (Lock-free)。

- 双缓冲机制 (Double Buffering)**：每个顶点维护**两个**邻居池 (`pool_A`，`pool_B`)。在一次迭代中，从 `pool_A` 读取当前邻居，计算出的新邻居写入 `pool_B`。迭代结束后，交换 `pool_A` 和 `pool_B` 的指针。这完全避免了读写冲突。

- **Warp 内原子操作：**对于更细粒度的更新（例如，将邻居的邻居添加到候选列表），可以使用 warp 内的原子操作（`atomicAdd`，`atomicCAS`），或利用 `shuffle` 指令进行高效的 warp 内通信和归约，以实现无锁更新。

4. GPU 并行设计与内核

4.1. 阶段 A：近似 kNN 图构建内核

`Kernel_NN_Descent_Iteration`

此内核完成一轮 NN-Descent 迭代。

- **线程/块映射：**
 - `Grid` 级别：覆盖所有 `N` 个顶点。
 - `Block` 级别：每个线程块负责处理 `B` 个顶点（例如 `B=256`）。
 - `Thread/Warp` 级别：一个 **Warp (32个线程)** 负责一个顶点的完整更新流程。这是保证高效协作和数据共享的核心。
- **执行流程 (Warp 级别)：**
 - 候选生成 (Candidate Generation)：**
 - Warp 内线程并行地从当前顶点的邻居池（`pool_A`）中，加载其邻居的邻居。
 - 使用共享内存作为临时候选缓冲区，并进行在线去重（可利用 warp 内 `ballot` 指令实现高效判重）。
 - 距离批量计算 (Batched Distance Computation)：**
 - Warp 内 32 个线程协作计算当前顶点与一个候选邻居之间的距离。每个线程负责计算部分维度，然后通过 `shuffle_down` 指令进行高效的 warp 内归约，得到最终距离。
 - 这个过程流水线化，高效计算完所有候选的距离。
 - 并行筛选与就地更新 (Parallel Filtering & Update)：**
 - 将新生成的候选邻居（及其距离）与当前邻居池（`pool_A`）中的邻居合并。
 - 在 warp 内，使用并行的 top-L 选择算法（如基于 bitonic sort 或 heap 的变体）从合并后的集合中选出新的 top-L 邻居。
 - 将结果写入 `pool_B`。
 - 乱序传播：**在步骤 1-3 中，候选的评估顺序应被随机化，以实现乱序传播。
- **收敛判定：**
 - 在每次迭代结束时，启动一个小型内核计算 `pool_A` 和 `pool_B` 之间的差异（被替换的邻居总数）。
 - 当差异小于 `N * update_threshold` 时，迭代停止。

4.2. 阶段 B：剪枝内核

Kernel_Prune_RNG

此内核执行 RNG 风格的剪枝。

- 线程/块映射：

- Grid 级别：覆盖所有 N 个顶点。
- Block 级别：每个线程块负责一个顶点 u 的剪枝。

- 执行流程 (Block 级别)：

a. 候选收集：将顶点 u 在近似 kNN 图中的所有邻居加载到共享内存。

b. 并行边筛选：

- 对于 u 的每一个邻居 v （即一条边 (u, v) ），启动一个 warp 来检查其是否应被剪枝。
- 该 warp 遍历 u 的其他所有邻居 w 。
- 并行计算 $\text{dist}(v, w)$ 。
- 如果发现存在任何一个 w 满足 $\text{dist}(v, w) < \text{dist}(u, v)$ ，则该 warp 投票决定 (u, v) 是冗余边，标记待删除。

c. 度上限控制：

- 在所有冗余边被标记后，如果 u 的剩余邻居数仍超过 $D_{\max}$ ，则简单地按距离截断，只保留最近的 $D_{\max}$ 个邻居。

4.3. 多 Stream 与异步 Pipeline

为了最大化 GPU 利用率，尤其是在处理多个组时，应采用多 Stream 策略。

- 数据拷贝与计算并行：

- Stream 1：cudaMemcpyAsync HtoD（将组 G1 的向量数据从主机拷贝到设备）。
- Stream 2：在组 G0 的数据上执行计算内核 (Kernel_NN_Descent_Iteration)。
- Stream 3：cudaMemcpyAsync DtoH（将组 G-1 的计算结果从设备拷回主机）。
- 通过这种方式，数据传输和计算可以重叠，隐藏 I/O 延迟。

- 组间并行：为每个要并行处理的组（或组的分块）分配一个独立的 CUDA Stream，使其计算内核可以在 GPU 上并发执行。

下一部分将介绍组间并行与负载均衡的具体策略。

5. 代码映射与实现规范

本章节旨在将前述高层设计方案与 `FilterVectorCode` 仓库中的现有 C++ 代码进行精确映射，为编码代理提供直接的、代码级的实现指导。核心目标是围绕 `ANNS::UniNavGraph::build_graph_for_all_groups` 函数，将其从一个基于 OpenMP 的多核 CPU 实现，重构为一个高效的 GPU 并行实现。

5.1. 目标函数静态分析: `build_graph_for_all_groups`

在 `UNG/codes/src/uni_nav_graph.cpp` 文件中，`build_graph_for_all_groups` 是组内 PG 构建的总入口。

5.1.1. 函数签名与职责

代码块

```
1 void ANNS::UniNavGraph::build_graph_for_all_groups();
2
```

- 职责:** 该函数负责遍历所有已划分的标签组 (`_num_groups`)，并为每一个组调用一个图构建算法（当前实现为 Vamana）来生成其内部的 Proximity Graph (PG)。
- 上下文:**
 - 调用前:** `prepare_group_storages_graphs()` 函数已执行完毕。这意味着：
 - 原始向量数据已根据标签组被重排 (`_base_storage` 已更新)。
 - 每个组的向量数据被封装在独立的 `_group_storages[group_id]` 中。
 - 每个组的图结构被初始化在 `_group_graphs[group_id]` 中。
 - 调用后:** 所有 `_group_graphs[group_id]` 中都填充了邻接列表，代表了各组的 PG。同时，`_vamana_instances[group_id]` 保存了每个组的 Vamana 索引实例，`_group_entry_points[group_id]` 保存了每个 PG 的入口点。

5.1.2. 内部数据结构与核心依赖

结构/类 (C++)	头文件	作用
<code>ANNS::UniNavGraph</code>	<code>uni_nav_graph.h</code>	UNG 索引的主类，管理所有数据和流程。 <code>build_graph_for_all_groups</code> 是其成员函数。
<code>_group_graphs</code> (vector)	<code>graph.h</code>	<code>std::vector<std::shared_ptr<Graph>></code> 。每个 <code>Graph</code> 对象代表一个组的邻接表 (<code>std::vector<IdxType>*</code>

		<code>neighbors</code>), 是构图的最终产出。
<code>_group_storages</code> (vector)	<code>storage.h</code>	<code>std::vector<std::shared_ptr<IStorage>></code> 。每个 <code>IStorage</code> 封装了一个组的向量数据, 提供 <code>get_vector()</code> 接口。
<code>_vamana_instances</code> (vector)	<code>vamana/vamana.h</code>	<code>std::vector<std::shared_ptr<Vamana>></code> 。存储每个组的 Vamana 索引对象。CPU 实现中, 图构建逻辑主要委托给 <code>Vamana::build</code> 方法。
<code>_group_id_to_range</code> (vector)	<code>uni_nav_graph.h</code>	<code>std::vector<std::pair<IdxType, IdxType>></code> 。记录每个 <code>group_id</code> 在重排后的 <code>_base_storage</code> 中对应的向量索引范围 <code>[start, end)</code> 。这对于数据拷贝至关重要。
<code>_distance_handler</code>	<code>distance.h</code>	<code>std::shared_ptr<DistanceHandler></code> 。提供统一的距离计算接口 <code>compute()</code> 。
<code>omp.h</code>	(System Header)	OpenMP 头文件, 用于 <code>#pragma omp parallel for</code> 并行化。

5.1.3. CPU 实现流程梳理

`build_graph_for_all_groups` 的当前 CPU 实现逻辑如下:

1. 并行循环: 使用 `#pragma omp parallel for schedule(dynamic, 1)` 对 `group_id` 从 1 到 `_num_groups` 进行并行遍历。 `schedule(dynamic, 1)` 意味着每个线程动态地获取一个 `group_id` 进行处理, 这是一种简单的负载均衡。
2. 小规模组快速路径:
 - `if (range.second - range.first <= _max_degree)`: 如果一个组的向量数量不大于图的最大度 `_max_degree`。
 - 调用 `build_complete_graph()`: 直接为该小组构建一个全连接图。这避免了在小规模数据上运行复杂图构建算法的开销。
3. 大规模组标准路径:
 - `else`: 对于规模较大的组。

- `_vamana_instances[group_id] = std::make_shared<Vamana>(false);`: 创建一个新的 Vamana 实例。
- `_vamana_instances[group_id]->build(...)`: 调用 Vamana 的核心 `build` 方法。这个方法内部实现了基于 NN-Descent 的迭代建图和剪枝逻辑（详见 `vamana.cpp` 中的 `link()` 和 `prune_neighbors()`）。这是我们需要用 GPU 实现替换的主要部分。

4. 入口点记录:

- `_group_entry_points[group_id] = _vamana_instances[group_id]->get_entry_point() + range.first;`: 在构图完成后，获取该组 Vamana 图的入口点，并将其从组内局部索引转换为全局索引后存储。

结论: GPU 优化的本质，就是将上述 `_vamana_instances[group_id]->build(...)` 的调用，替换为我们设计的两阶段 GPU 并行方案。

5.2. 函数级实现蓝图: `build_graph_for_all_groups` 的 GPU 重构

重构的目标是保留 `build_graph_for_all_groups` 的函数签名和外部行为，但将其内部实现替换为一套高效的、由调度器驱动的 GPU 内核调用流程。

5.2.1. 新增/替换的 GPU 模块接口原型

以下是为实现 GPU 并行化需要新增的核心 CUDA/C++ 接口伪签名。

代码块

```
1  // ===== GPU 端核心内核 =====
2
3  // 阶段 A: 执行一轮 NN-Descent 迭代
4  __global__ void kernel_nn_descent_iteration(
5      float* d_vectors,           // 设备端所有向量数据 (SoA 布局)
6      int* d_neighbor_pool_A,     // 设备端邻居池 (读)
7      int* d_neighbor_pool_B,     // 设备端邻居池 (写)
8      float* d_distance_cache_A,  // 设备端距离缓存 (读)
9      float* d_distance_cache_B,  // 设备端距离缓存 (写)
10     int* d_group_starts,        // 各组在 d_vectors 中的起始索引
11     int* d_group_sizes,         // 各组的大小
12     int group_id,               // 当前内核处理的组 ID
13     int L,                      // 邻居池长度
14     int D                       // 向量维度
15 );
16
17 // 阶段 B: 执行 RNG 风格的图剪枝
18 __global__ void kernel_prune_rng(
19     float* d_vectors,           // 设备端向量数据
20     int* d_knn_graph,           // 设备端近似 kNN 图的邻接表 (来自阶段A)
21     int* d_final_pg,            // 设备端最终剪枝后的 PG (输出)
```

```

22     int*    d_group_starts, // 各组的起始索引
23     int*    d_group_sizes, // 各组的大小
24     int     group_id,
25     int     L,              // kNN 图的邻居数
26     int     D_max,          // 最终 PG 的最大度
27     float   alpha           // 剪枝参数 alpha
28 );
29
30
31 // ===== CPU 端调度与封装 =====
32
33 // GPU 资源管理器
34 class GpuResourceManager {
35 public:
36     // 管理 CUDA Stream、设备内存分配/释放等
37     cudaStream_t get_stream();
38     void*        malloc_device(size_t bytes);
39     void         free_device(void* d_ptr);
40 };
41
42 // 组内图构建的 GPU 任务封装
43 class GpuPgBuilder {
44 public:
45     // 构造函数，初始化必要的 GPU 资源
46     GpuPgBuilder(std::shared_ptr<GpuResourceManager> res_manager);
47
48     // 主执行函数，完成单个组的 PG 构建 (阶段 A + B)
49     void build_group_pg(
50         int     group_id,
51         const std::pair<IdxType, IdxType>& range, // 组的向量范围
52         char*   h_all_vectors,                  // 主机端所有向量数据指针
53         IdxType D,                              // 维度
54         IdxType L_build,                        // 阶段A邻居池长度
55         IdxType max_degree,                    // 阶段B最大度
56         float   alpha,                        // 剪枝 alpha
57         Graph&  out_graph                      // 输出的图结构 (主机端)
58     );
59 };
60
61 // 重构后的 build_graph_for_all_groups 函数体
62 void UniNavGraph::build_graph_for_all_groups_gpu_version() {
63     // 1. 初始化 GPU 资源管理器和 PG 构建器
64     auto gpu_res_manager = std::make_shared<GpuResourceManager>();
65     GpuPgBuilder pg_builder(gpu_res_manager);
66
67     // 2. 创建任务队列
68     std::vector<int> group_tasks;

```

```

69     for (int gid = 1; gid <= _num_groups; ++gid) {
70         group_tasks.push_back(gid);
71     }
72
73     // 3. 实现一个简单的动态调度器 (或使用 TBB/OpenMP 的 task)
74     // 多线程并行地从 group_tasks 中取任务并执行
75     #pragma omp parallel for schedule(dynamic, 1)
76     for (int i = 0; i < group_tasks.size(); ++i) {
77         int group_id = group_tasks[i];
78
79         // 小规模组快速路径 (CPU)
80         const auto& range = _group_id_to_range[group_id];
81         if (range.second - range.first <= _max_degree) {
82             build_complete_graph(_group_graphs[group_id], range.second -
range.first);
83         } else {
84             // 大规模组标准路径 (GPU)
85             pg_builder.build_group_pg(
86                 group_id,
87                 range,
88                 _base_storage->get_vector(0), // 获取整个数据块的指针
89                 _base_storage->get_dim(),
90                 _lbuild,
91                 _max_degree,
92                 _alpha,
93                 *_group_graphs[group_id] // 传入引用
94             );
95         }
96
97         // 注意: 入口点选择逻辑需要根据 GPU 实现调整
98         // 可以在 pg_builder.build_group_pg 内部计算并返回
99         // 或者在所有组完成后, 再批量计算
100        // _group_entry_points[group_id] = ...;
101    }
102
103    // 4. 最终需要将所有 group 的邻接表ID从局部转换为全局
104    add_offset_for_uni_nav_graph();
105 }
106

```

5.2.2. GPU 并行策略与内核实现要点

kernel_nn_descent_iteration (阶段A):

- 线程映射: 一个 Warp (32 线程) 处理一个顶点 `p` 的一轮更新。
- 数据流:

- a. **加载:** Warp 内线程并行从全局内存加载 `p` 的当前 `L` 个邻居的 ID 到共享内存 `sh_neighbors`。
- b. **候选生成:** 每个线程负责 `sh_neighbors` 中的一个子集。对于邻居 `q`，加载 `q` 的 `L` 个邻居，形成 `p` 的候选集。使用原子操作 `atomicCAS` 或 `warp shuffle (__shfl_sync)` 在共享内存中对候选集进行去重和合并。
- c. **距离计算:** Warp 内 32 个线程协作，并行计算 `p` 与一个候选点 `c` 之间的距离。每个线程计算 `D/32` 个维度的部分和，然后用 `__shfl_down_sync` 进行 warp 内归约，快速得到总距离。
- d. **并行筛选:** 将 `p` 的旧邻居和所有新候选（连同距离）放在共享内存中。使用 warp 级的并行排序算法（如奇偶排序或 Bitonic sort 的变体）或 top-k 选择算法，找出新的 `L` 个最近邻。
- e. **写回:** 将新的 `L` 个邻居及其距离写回到 `d_neighbor_pool_B` 和 `d_distance_cache_B`。

`kernel_prune_rng` (阶段 B):

- **线程映射:** 一个 Block 处理一个顶点 `u` 的剪枝。
- **数据流:**
 - a. **加载:** Block 内所有线程将 `u` 的 `L` 个邻居（来自阶段 A 的 kNN 图）及其向量数据加载到共享内存。
 - b. **并行边筛选:**
 - 对于 `u` 的每个邻居 `v`，分配一个 Warp。
 - 该 Warp 遍历 `u` 的所有其他邻居 `w`。
 - Warp 内协作计算 `dist(v, w)`。
 - 如果存在任何 `w` 使得 `dist(u, w) < dist(u, v)` 且 `dist(v, w) < dist(u, v)`，则该 Warp 通过 `__ballot_sync` 投票，如果至少有一票，则边 `(u, v)` 被标记为冗余。
 - c. **度约束:** 统计所有非冗余的边。如果数量超过 `D_max`，则按距离排序，只保留最近的 `D_max` 条。
 - d. **写回:** 将最终的邻居列表写入 `d_final_pg`。

5.2.3. 组间并行与负载均衡

- **代价估计:** 在调度前，可简单根据组的大小 `(range.second - range.first)` 对任务进行成本估算。大组优先调度。
- **动态调度:** CPU 端的 `#pragma omp parallel for schedule(dynamic, 1)` 是一个有效的基线。多个 CPU 线程充当生产者，不断向 GPU 任务队列（由 `GpuPgBuilder` 实例表示）提交任务。

- **Multi-GPU:** 如果有多个 GPU，可以将 `group_tasks` 均分到不同的 GPU 上。每个 GPU 拥有自己的 `GpuResourceManager` 和 `GpuPgBuilder` 实例。
- **Work Stealing:** 对于更高级的负载均衡，当一个 GPU 的任务队列为空时，其关联的 CPU 调度线程可以尝试从其他更繁忙的 GPU 的任务队列中“窃取”一个任务来执行。
- **大组分块 (Tiling):** 对于规模超过特定阈值（如 100,000 点）的超大组，可以将其虚拟地切分为多个子块 (tile)。每个 tile 被视为一个独立的构建任务，可以被调度到不同的 Stream 甚至不同 GPU 上，从而实现组内并行。最后通过一个额外的合并内核将各 tile 的结果融合。

5.2.4. 参数与可调开关

参数	建议默认值	调优建议
<code>k</code> (查询邻居数)	10-100	由应用场景决定，通常无需在构图时调整。
<code>L</code> (邻居池长度)	<code>k + 50</code>	越大，kNN 图质量越高，但计算和显存开销也越大。建议设为 <code>k</code> 的 1.5-2 倍或一个固定较大的值（如 100-200）。
<code>C</code> (候选规模)	64-128	影响每次迭代能探索的新邻居数量。太小则收敛慢，太大则共享内存压力大。
迭代上限	10-20	防止无法收敛的组无限循环。通常 5-10 轮内即可收敛。
更新阈值	0.001	每轮迭代中，邻居更新比例低于此值则停止。
剪枝度阈值 <code>D_max</code>	32-64	最终 PG 的最大出度。影响图的稀疏度和搜索性能。与 <code>max_degree</code> 保持一致。
CUDA Stream 数量	4-8	太少无法有效隐藏延迟，太多则调度开销变大。通常与硬件的 Copy/Compute Engine 数量匹配。

5.2.5. 端到端伪代码

伪代码 1: 构建单个组 PG (`GpuPgBuilder::build_group_pg`)

代码块

```

1  function build_group_pg(group_info):
2      // 1. 在 GPU 上为该组分配内存
3      d_vectors = GpuResourceManager.malloc(...)
4      d_pool_A, d_pool_B = GpuResourceManager.malloc(...)
5      d_dist_A, d_dist_B = GpuResourceManager.malloc(...)
6      d_final_pg = GpuResourceManager.malloc(...)
7
8      // 2. 异步拷贝数据到 GPU
9      stream = GpuResourceManager.get_stream()
10     cudaMemcpyAsync(d_vectors, h_vectors + group_info.start, ..., stream)
11
12     // 3. 初始化随机邻居池
13     kernel_random_init_pool<<<...>>>(d_pool_A, d_dist_A, ...)
14
15     // 4. 阶段 A: 迭代构建近似 kNN
16     for i in 0..MAX_ITERATIONS:
17         kernel_nn_descent_iteration<<<...>>>(..., d_pool_A, d_pool_B, ...)
18         swap(d_pool_A, d_pool_B)
19
20         // 检查收敛 (可异步执行)
21         updates = check_convergence(...)
22         if updates / group_info.size < CONVERGENCE_THRESHOLD:
23             break
24
25     // 5. 阶段 B: 剪枝
26     kernel_prune_rng<<<...>>>(..., d_pool_A, d_final_pg, ...)
27
28     // 6. 异步拷贝结果回主机
29     cudaMemcpyAsync(h_output_graph, d_final_pg, ..., stream)
30
31     // 7. 同步并释放资源
32     cudaStreamSynchronize(stream)
33     GpuResourceManager.free_all(...)
34

```

伪代码 2: 所有组的并行调度 (`build_graph_for_all_groups_gpu_version`)

代码块

```

1  function build_graph_for_all_groups_gpu_version():
2      // 1. 初始化 GPU 管理器和多个 GpuPgBuilder (每个对应一个 CPU 调度线程)
3
4      // 2. 创建所有组的任务列表
5      all_tasks = generate_task_list(1..num_groups)
6
7      // 3. 使用 OpenMP 或 TBB 等并行框架

```

```

8  parallel_for each task in all_tasks:
9      builder = get_thread_local_builder() // 获取线程自己的 builder
10
11     // 如果组太小, 用 CPU 处理
12     if task.size <= CPU_THRESHOLD:
13         cpu_build_complete_graph(task)
14     else:
15         // 否则调用 GPU 构造函数
16         builder.build_group_pg(task)
17
18 // 4. 等待所有任务完成
19
20 // 5. 执行全局ID转换
21 add_offset_for_uni_nav_graph()
22

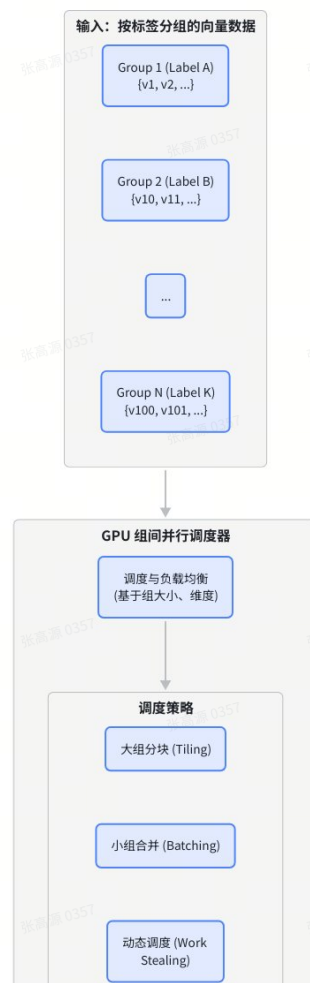
```

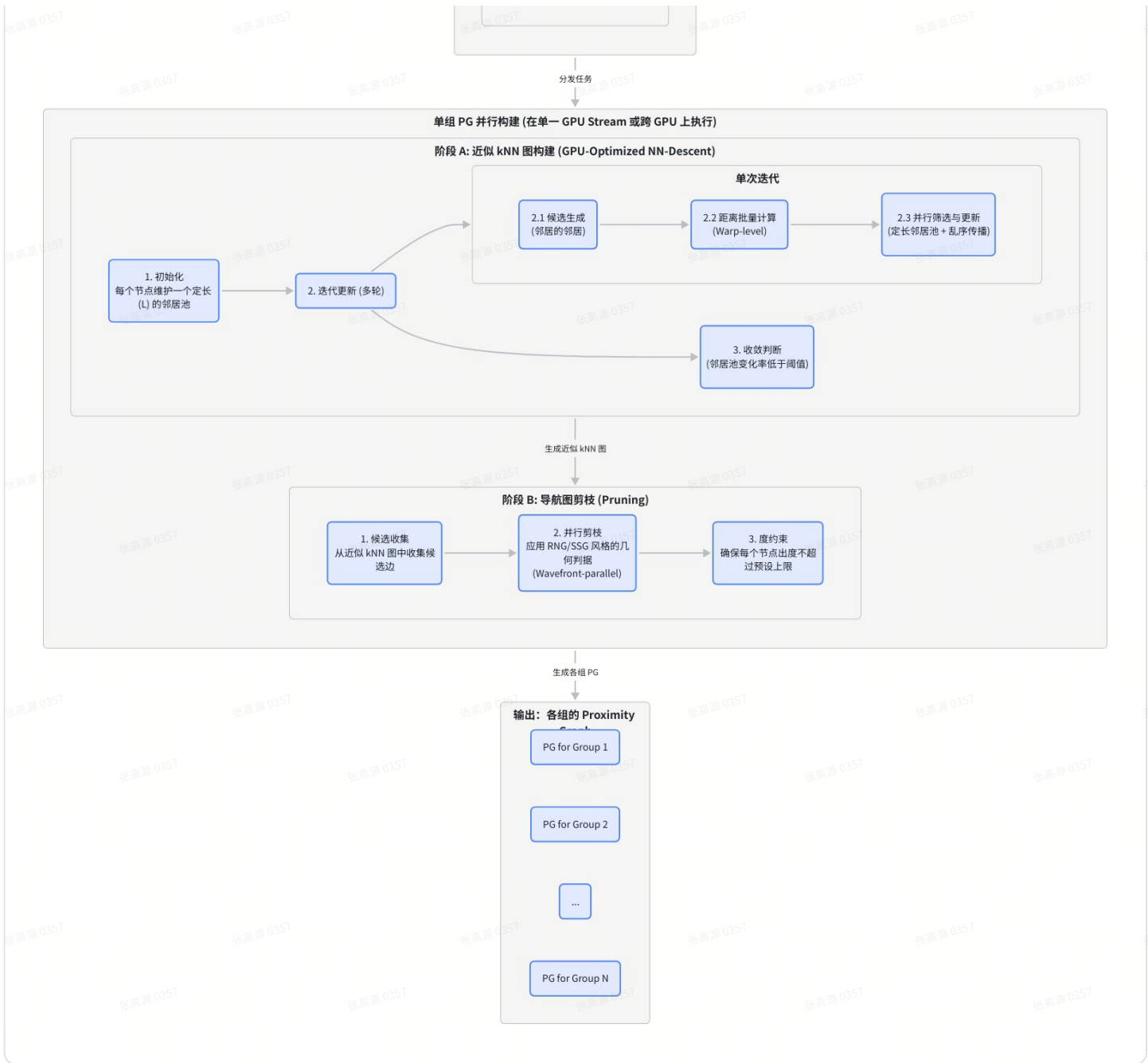
5.3. 可视化流程

为了更直观地理解整个流程，以下提供两个核心图示。

5.3.1. 端到端实现流程图

此图描绘了从接收分组数据到最终生成所有组 PG 的完整端到端流程。





5.3.2. GPU 调度与负载均衡示意图

此图展示了在多 GPU 环境下，如何通过任务分块、批处理和工作窃取等策略实现高效的负载均衡。



